# 1 - Computing Infrastructures Fundamentals

Computing Infrastructures

Unat Teksen

April 3, 2026

## Contents

**1 What is a Computing Infrastructure? 3**
1.1 Three Key Elements of the Definition . . . 3

**2 The Spectrum of Computing Infrastructures 3**
2.1 RAM vs. Computation Diagram . . . 3
2.2 Infrastructure Types Comparison Table . . . 3
2.3 Overview of Each Infrastructure Type . . . 3
2.3.1 Internet of Things (IoT) . . . 4
2.3.2 Embedded Devices / Embedded PCs . . . 4
2.3.3 Personal Computer (PC) . . . 4
2.3.4 Edge Computing Systems . . . 4
2.3.5 Data Centers . . . 4

**3 Course Structure Overview 5**

**4 Data Centers — Structure and Technology 5**
4.1 Physical Structure of a Data Center . . . 5

**5 Virtualization: Virtual Machines and Containers 6**
5.1 The Problem Virtualization Solves . . . 6
5.2 Virtual Machine Monitors (VMMs / Hypervisors) . . . 6
5.3 Containers (e.g., Docker) . . . 7
5.4 VM vs. Container — Critical Comparison Table . . . 7

**6 The Data Center Dilemma: Build, Rent, or Buy? 8**
6.1 Option 1: Enterprise Data Center (Build Your Own) . . . 8
6.2 Option 2: Colocation Data Center . . . 9
6.3 Option 3: Managed Hosting / Third Party Data Center . . . 10
6.4 Option 4: Cloud Computing . . . 10

**7 Data Center Costs and Environmental Impact 11**
7.1 Installed Power . . . 11
7.2 Water Consumption . . . 11
7.3 Electricity Consumption . . . 12

7.4 Amortized Cost Breakdown . . . . . 12
7.5 Advantages and Disadvantages of Data Centers . . . . . 12

**8 Edge / Fog Computing Systems 13**
8.1 Why Edge Computing Exists . . . . . 13
8.2 The Three-Layer Architecture (Pyramid Model) . . . . . 14
8.3 Real Example 1: Edge Computing in Manufacturing . . . . . 15
8.4 Real Example 2: Wind Turbine Blade Inspection (UAV-based) . . . . . 15
8.5 Edge/Fog Computing Hardware — Real Specifications . . . . . 16

**9 Embedded PCs 18**
9.1 What Are Embedded PCs? . . . . . 18
9.2 Key Examples from the Slides . . . . . 18
9.3 Advantages and Disadvantages . . . . . 19
9.4 When to Use Which Infrastructure Type? . . . . . 19

**10 Internet of Things (IoT) 19**
10.1 Definition . . . . . 20
10.2 Key Examples from the Slides . . . . . 20
10.3 Arduino Nano Family Specification Comparison . . . . . 21
10.4 Advantages and Disadvantages of IoT . . . . . 21

**11 The IT Perspective for Computing Infrastructures 22**
11.1 The Three-Layer IT Model . . . . . 22
11.2 Mapping to Course Topics . . . . . 23

**12 Complete Spectrum Summary — The Big Picture 24**

**13 Master Exam Focus Summary — Lecture 1 24**

# 1 What is a Computing Infrastructure?

**Definition**

A **computing infrastructure** is a technological infrastructure that provides **hardware** and **software** for computation to **other systems and services**.

## 1.1 Three Key Elements of the Definition

- **Hardware** — physical machines, processors, storage, network cards
- **Software** — operating systems, middleware, virtualization layers
- **Serves others** — not self-contained; it exists to support applications and services running on top of it

**Note**

A computing infrastructure is like the electrical grid: you don't think about it, but everything plugs into it. It is a *platform* that other systems depend on, not an end in itself.

# 2 The Spectrum of Computing Infrastructures

## 2.1 RAM vs. Computation Diagram

The course uses a 2D diagram to classify infrastructure types:

- **X-axis:** Computation Speedup (relative to a standard PC = 1x)
- **Y-axis:** RAM (relative to a standard PC = 1x)
- **PC** sits at the origin as the reference point (1x, 1x)

## 2.2 Infrastructure Types Comparison Table

| maingreen!20 **Infrastructure** | **Computation Speedup** | **RAM** |
|---|---|---|
| Internet of Things (IoT) | 0.0001x – 0.0005x | 0.01x – 0.05x |
| Embedded Devices | 0.05x – 0.1x | 0.1x – 0.5x |
| Embedded PCs | 0.05x – 0.1x | 0.1x – 0.5x |
| **PC (Reference)** | **1x** | **1x** |
| Edge Computing Systems | 20x – 50x | 50x – 100x |
| Data Center | 100x – 1000x | Very high |

## 2.3 Overview of Each Infrastructure Type

### 2.3.1 Internet of Things (IoT)

- Tiny, extremely constrained devices (e.g., Arduino, smart thermostat, soil sensor)
- Very little RAM — kilobytes, not gigabytes
- Very slow processors
- Primary purpose: **sense the physical world and transmit small amounts of data**
- They do *not* process data — they collect and relay it

### 2.3.2 Embedded Devices / Embedded PCs

- Slightly more capable microcontrollers (e.g., STM32, ARM Cortex chips)
- Used in industrial controllers, medical devices, automotive systems
- Can run small real-time operating systems
- Embedded PCs (e.g., Raspberry Pi, BeagleBone) are programmable like normal PCs in a tiny, low-power form factor

### 2.3.3 Personal Computer (PC)

- General-purpose, programmable
- Used as the **baseline reference** (1x) in the spectrum diagram

### 2.3.4 Edge Computing Systems

- Server-class hardware deployed *close to where data is generated*
- Deployed in factories, hospitals, vehicles — not in remote data centers
- 20–50x computation and 50–100x RAM compared to a PC
- Key idea: **proximity** — processing happens near the data source to reduce latency

### 2.3.5 Data Centers

- Most powerful class: 100x–1000x computation of a PC
- Tens of thousands of servers working together
- Used for cloud services, AI training, large-scale databases
- Physically large, consume enormous power, require specialized cooling and management

**★ Exam Focus**

- **Memorize the definition** of computing infrastructure (slides exact wording)
- **Know the spectrum** — place each type on the RAM vs. Computation axes with approximate multipliers
- **PC is the reference point** (1x, 1x) — all other values are relative to it
- Understand *why* different infrastructures exist: it is a trade-off between power, energy, cost, latency, and location

# 3 Course Structure Overview

- **A. Hardware (HW) Infrastructures** (February – mid March)
  - System-level: Computing Infrastructures, Data Center Architectures, Rack/Structure
  - Node-level: Server (computation, HW accelerators), Storage (type, technology), Networking (architecture and technology)
  - Building-level: Cooling systems, power supply, failure recovery
- **B. Software (SW) Infrastructures** (End of May)
  - Virtualization: Process/System VM, Hypervisor, Para/Full virtualization, Containers
  - Computing Architectures: Cloud Computing (types, characteristics), X-as-a-service, Edge/Fog Computing
  - Machine and deep learning-as-a-service
- **C. Methods** (Mid March – Mid May)
  - Reliability and availability of datacenters (definitions, fundamental laws, RBDs)
  - Disk performance (Type, Performance, RAID)
  - Scalability and performance of datacenters (definitions, fundamental laws, queuing network theory)

# 4 Data Centers — Structure and Technology

## 4.1 Physical Structure of a Data Center

A data center houses large numbers of servers organized for three main functions:

- **Server for Processing** — CPUs, GPUs doing actual computation
- **Server for Storage** — storing massive amounts of data (databases, file systems)

- **Server for Communication** — networking equipment handling data flow between systems and the outside world

**Note**

These three categories work together — computation needs data (storage), and results need to be delivered (communication). You cannot have one without the others.

- Data centers come in very different physical forms:
  - Architecturally impressive (e.g., Pionen White Mountains in Stockholm — built inside a Cold War bunker carved into a mountain)
  - Purely industrial — massive warehouse-like buildings surrounded by cooling towers and generators
- **What matters is what is inside, not how it looks outside**

# 5 Virtualization: Virtual Machines and Containers

## 5.1 The Problem Virtualization Solves

- Physical servers are expensive
- Dedicating one physical machine per application leaves most of the machine idle and wasted
- Solution: **virtualization** — run multiple isolated workloads on the same physical hardware

## 5.2 Virtual Machine Monitors (VMMs / Hypervisors)

**Definition**

A **Virtual Machine Monitor (VMM)** or **Hypervisor** is a software layer that sits between physical hardware and guest operating systems, managing and isolating multiple VMs. Each VM gets a complete operating system (Guest OS) and behaves as if it has dedicated hardware.

**Stack (bottom to top):**

- `INFRASTRUCTURE` (physical hardware)
- `HOST OPERATING SYSTEM`
- `HYPERVISOR`
- `[GUEST OS | GUEST OS | GUEST OS]`
- `[BINS/LIBS | BINS/LIBS | BINS/LIBS]`
- `[APP 1  | APP 2  | APP 3  ]`

- Each VM is completely unaware of other VMs
- Can run Windows VM and Linux VM on the same physical machine simultaneously
- **Pros:** Strong isolation, full OS flexibility
- **Cons:** Heavy — each VM carries a full OS, consuming significant RAM and CPU overhead; slow startup (minutes)

## 5.3 Containers (e.g., Docker)

**Definition**

**Containers** package applications with all their dependencies into a standardized unit. They **share the host OS kernel** — there is no Guest OS per application.

**Stack (bottom to top):**

- `INFRASTRUCTURE` (physical hardware)
- `HOST OPERATING SYSTEM`
- `DOCKER ENGINE`
- `[BINS/LIBS | BINS/LIBS | BINS/LIBS]`
- `[APP 1  | APP 2  | APP 3  ]`
- **Pros:** Lightweight, fast startup (seconds), much lower overhead than VMs
- **Cons:** Less isolation — all containers share the same kernel, so a kernel-level vulnerability affects all containers

## 5.4 VM vs. Container — Critical Comparison Table

| maingreen!20 **Feature** | **Virtual Machine** | **Container** |
|---|---|---|
| OS per app | Yes (Guest OS) | No (shared host OS) |
| Isolation level | Strong | Moderate |
| Startup time | Minutes | Seconds |
| Resource overhead | High | Low |
| Use case | Full OS flexibility | Microservices, rapid deployment |
| Managed by | Hypervisor | Container Engine (e.g., Docker) |

**★ Exam Focus**

- **Know both stack diagrams** — be able to draw and explain each layer
- **Key difference:** VMs have Guest OS, containers share host OS kernel
- **Know the trade-offs:** isolation vs. overhead, startup time, resource usage
- This topic is covered in detail later in the SW Infrastructures section but introduced here because data centers use both extensively

# 6 The Data Center Dilemma: Build, Rent, or Buy?

**Note**

**Fundamental business context:** Data centers exist as part of a business, not as ends in themselves. A business makes money from its service, not from owning servers. Therefore: *how much capital and expertise should be spent on infrastructure vs. core business?*

## 6.1 Option 1: Enterprise Data Center (Build Your Own)

**Definition**

An **Enterprise Data Center** is a facility wholly built, maintained, operated, and managed by the company providing the target service.

**Typical facility requirements:**

- Highly secure site (IT security and physical building security)
- Redundant electrical, current-conditioned, power distribution systems
- Redundant climate control systems
- Raised-floor system centers integrated with multiple fire suppression systems
- Office space as required
- Complex network infrastructure: gateways, routers, data cables
- Multiple server complexes and data storage systems
- Centrally-located system monitors throughout the facility

**Pros:**

- The data center is a **company asset** (financial value on the balance sheet)
- **Complete control** over all operational activities

**Cons:**

- **Very high capital investment** upfront (tens to hundreds of millions of euros)
- Requires **specialized expertise** to design and operate
- Dedicated teams of DC engineers, network specialists, and security experts needed

**Who uses this?** Large enterprises: banks, government agencies, telecoms — organizations where data control and security are paramount.

## 6.2 Option 2: Colocation Data Center

**Definition**

A **Colocation Data Center** is a large facility built to accommodate multiple businesses. The business rents its own physical space (cabinet, cage, or dedicated room) inside the provider's data center and can fill it with proprietary IT equipment or rent equipment from the co-locator.

**Pros:**

- Still **high control** on the IT equipment and operational environment
- **No need for DC design skills** — provider handles the facility
- **Lower capital investment** — you are not building a building
- **Shared costs** for high-quality infrastructure (power, cooling, security)
- Professional staff dedicated to guarantee operational quality

**Cons:**

- The data center is **no longer a company asset**
- **Restricted maintenance access** — may need to schedule visits by time or resources
- **IT equipment purchases and upgrades** are still your responsibility

**Note**

A real colocation provider guarantee: *"We guarantee **99.995% uptime** for your data center facilities."* — This means roughly **26 minutes of downtime per year** ("Five Nines" reliability standard — you will study this in the Methods section).

## 6.3 Option 3: Managed Hosting / Third Party Data Center

**Definition**

In **Managed Hosting**, the business does not own or control the data center or any space. The customer rents IT equipment and infrastructure from a third-party provider. Resources are **not shared** — this is still a **single-tenant solution**.

**Pros:**

- **Low capital investment** — pay operationally for what you use
- Significant **cost savings for small businesses**
- **No expertise needed** to design or run the data center

**Cons:**

- **Higher operational costs** — ongoing fees may exceed cost of ownership over time
- **Lower control** over operational activities
- **Customer lock-in problem** — once deeply integrated with one provider, migrating away is extremely difficult and expensive

## 6.4 Option 4: Cloud Computing

Cloud solutions are **additional options** beyond the three physical infrastructure approaches.

**The cloud deployment spectrum — what YOU manage:**

| maingreen!20 **Model** | **What YOU manage** |
|---|---|
| On-Premises IT | Everything: Data, Apps, DBs, OS, Virtualization, Physical Servers, Storage, Networks, Data Center |
| Colocation | Data, Apps, DBs, OS, Virtualization, Physical Servers |
| Hosting | Data, Apps, DBs, OS |
| IaaS | Data, Apps, DBs, OS |
| PaaS | Data, Apps |
| SaaS | Data only |

- As you move from On-Premises → SaaS: **you manage less, provider manages more**
- Trade-off: **control vs. convenience**

**Cloud service model examples:**

- **IaaS** (Infrastructure as a Service): AWS EC2 — you get virtual machines, you install your own OS and applications

- **PaaS** (Platform as a Service): Google App Engine — you deploy code, platform manages OS and runtime
- **SaaS** (Software as a Service): Gmail, Office 365 — you just use the application, everything else is hidden

**★ Exam Focus**

- **Know all four deployment options** and their pros/cons — highly likely as a comparison question
- **Know the cloud spectrum**: On-premises → Colocation → Hosting → IaaS → PaaS → SaaS
- **Know what you manage at each level** — the table above is critical
- **Single-tenant vs. multi-tenant**: Managed Hosting is single-tenant; Cloud (IaaS/PaaS/SaaS) is typically multi-tenant
- **99.995% uptime = Five Nines** — know what this means in terms of downtime per year (≈ 26 minutes)
- **Lock-in problem** — a key disadvantage of managed hosting and some cloud providers

# 7 Data Center Costs and Environmental Impact

## 7.1 Installed Power

- Global data center installed power grew from approximately **30 GW in 2010 to nearly 60 GW by 2020**
- Growth regions: North America, Europe, Asia Pacific, Latin America, MEA
- Growth is **accelerating** due to AI workloads
- **1 GW = 1,000 MW = 1,000,000,000 Watts**
- A large nuclear power plant produces approximately 1 GW
- Global data centers consume the equivalent of many dozens of nuclear power plants

## 7.2 Water Consumption

- Data centers use enormous amounts of water for **evaporative cooling** (water evaporates to remove heat from servers)
- A **15-megawatt data center** uses approximately **130 million gallons of water per year**
- This is equivalent to:
  - 3 hospitals (130M gallons)

  - 100 acres of almond orchard (115M gallons)
  - 2 eighteen-hole golf courses (100M gallons)
- Data center location is partly driven by access to water and cold climate
- Examples of cool-climate locations: Scandinavia, Ireland, Pacific Northwest

## 7.3 Electricity Consumption

- US data center electricity consumption projections (2014–2028):
  - **2014:** ≈1.9% of total US electricity (≈70 TWh)
  - **Future range:** 4.4% to 6.7–12.0% of US total (massive increase driven by AI and cloud)
- Energy efficiency is a major **engineering and economic concern**, not just environmental
- Every watt saved = money saved

## 7.4 Amortized Cost Breakdown

| maingreen!20 **Component** | **% of Amortized Cost** | **Sub-components** |
|---|---|---|
| Servers | ∼45% | CPU, memory, disk |
| Infrastructure | ∼25% | UPS, cooling, power distribution |
| Power draw | ∼15% | Electrical utility costs |
| Network | ∼15% | Switches, links, transit |

- **Servers are the biggest single cost** (∼45%)
- Infrastructure + Power together = 40% of costs — this is why cooling and power efficiency are central engineering problems

**Definition**

**UPS (Uninterruptible Power Supply):** Battery systems that keep servers running during a power grid failure until generators can start up.

## 7.5 Advantages and Disadvantages of Data Centers

**Advantages:**

- Lower IT costs (shared infrastructure, economies of scale)
- High performance (massive parallelism)
- Instant software updates

- "Unlimited" storage capacity
- Increased data reliability (redundancy)
- Universal document access (from anywhere)
- Device independence (access from any device)

**Disadvantages:**

- Require a constant internet connection
- Performance degrades with slow connections
- Hardware features might be limited (you use what the provider offers)
- Privacy and security concerns (data on someone else's hardware)
- High power consumption (environmental and cost issue)
- **Latency in making decisions** — this is the critical driver for Edge Computing

**✓ Main Takeaway**

The last disadvantage — **latency** — is the key motivation for the entire Edge Computing paradigm. When decisions must be made in microseconds to milliseconds, a round-trip to a remote data center is physically impossible to achieve in time.

**★ Exam Focus**

- **Cost breakdown**: 45% servers, 25% infrastructure, 15% power, 15% network — memorize these values
- **Latency as the key disadvantage** of centralized data centers — this motivates Edge Computing
- **Water consumption**: a 15 MW DC uses 130M gallons/year
- **Power growth**: 30 GW (2010) → 60 GW (2020), growing rapidly

# 8 Edge / Fog Computing Systems

## 8.1 Why Edge Computing Exists

- **Root cause:** latency in data center / cloud processing
- Even at the speed of light over fiber, a round-trip to a remote data center takes **tens to hundreds of milliseconds**
- In industrial control, medical devices, or autonomous vehicles, this delay can be **catastrophic**
- Edge Computing brings computation **physically close to where data is generated**

**Example:** A robot arm on a factory floor detects a dangerous situation. If it must send data to the cloud and wait for a response, the robot may have already caused damage before the stop command arrives.

## 8.2 The Three-Layer Architecture (Pyramid Model)

This is the canonical architectural model for Edge/Fog/Cloud systems:

| maingreen!20 **Layer** | **Description** | **Timescale** |
|---|---|---|
| **Cloud** (top) | Centralized, high power, highly elastic virtualized compute/storage/network | Minutes to hours |
| **Fog / Edge Servers** (middle) | On-premises hardware, more computationally intensive processing | 100s of ms to minutes |
| **IoT & AI-Enabled Edge Sensors** (bottom) | Data acquisition and partial processing at the network edge | Microseconds to milliseconds |

- **Bottom Layer — IoT & AI-Enabled Edge Sensors:**
  - Data acquisition at the physical world boundary
  - Simple filtering, basic anomaly detection
  - Operates at **microsecond to millisecond** timescales
  - Example: a temperature sensor that detects "this is abnormal" without sending raw data
- **Middle Layer — Edge/Fog Servers:**
  - On-premises hardware performing more computationally intensive processing
  - Runs heavier algorithms on pre-filtered data from sensors
  - Operates at **100s of milliseconds to minutes** timescale
  - Example: a server in a factory running a machine learning model for quality inspection
- **Top Layer — Cloud:**
  - Virtualized computing, storage, and network resources with highly elastic capacity

  - Handles long-term data storage, model training, large-scale analytics
  - Operates at **minutes to hours** timescale
  - Not suitable for real-time decisions; ideal for learning and global optimization

**Note**

**Key principle:** Process data at the **lowest layer possible** to minimize latency, but push data **upward** when you need more computation or long-term storage.

**Key design pattern:** *Learning in the Cloud, Deploying in the Fog* — Cloud trains complex AI models using large datasets (time-insensitive), Fog deploys simplified versions for real-time inference.

## 8.3 Real Example 1: Edge Computing in Manufacturing

- **Goal:** Hierarchical data acquisition and control cycle with a common software paradigm
- **Cloud:** Trains complex AI models using large datasets
- **Fog:** Deploys simplified models for real-time inference; fast reaction to rich local analysis
- **IoT Endpoints:** Collect raw sensor data; data objectization and reduction
- Latency requirements per level:
  - IoT endpoints: $\mu$secs — direct sensor/actuator control
  - Fog layer: 100s of $\mu$secs to msecs — fast local reaction
  - Cloud: 100s of msecs to minutes — learning, optimization, reporting

## 8.4 Real Example 2: Wind Turbine Blade Inspection (UAV-based)

This example compares **Cloud-based solution vs. Edge solution** for wind turbine inspection:

**Cloud-Based Solution (old approach):**

- UAV flies and captures images
- Data is **manually uploaded** to the cloud
- Cloud processes data (damage detection, classification)
- Results come back later
- **Problem:** No immediate feedback to UAV operator; bad images only discovered after landing; mission must be repeated

**Edge Solution (new approach) — Pipeline stages:**

1. **Data Capture** — UAV collects images
2. **Quality Assessment** — on-site edge device immediately assesses image quality; real-time feedback to UAV operator
3. **Blade Part Detection** — identifies which part of blade is in image (tip, root, leading edge, etc.)
4. **Damage Detection / Clearing House** — on-site processing filters and compresses relevant data
5. **Damage Classification** — cloud processes final results
6. **Severity Assessment** — final report with damage severity (S1, S2 severity levels)

- The edge device processes data **immediately on-site**
- Operator knows in real-time if a retake is needed
- Output: JSON with bounding boxes, damage classification, severity, blade part identification

**✓ Main Takeaway**

**Key insight:** Not all processing needs to happen in the cloud. The **split between edge and cloud is a design decision** based on latency requirements, data volume, and connectivity. The edge handles time-critical steps; the cloud handles complex analytics.

## 8.5 Edge/Fog Computing Hardware — Real Specifications

Example Dell PowerEdge servers used as edge computing systems:

| maingreen!20 **Model** | **Rack Units** | **Max Memory** | **Max Storage** | **GPU** |
|---|---|---|---|---|
| PowerEdge XE9680 | 6U | 32×128 GB DDR5 (4 TB) | 16× E3.S or 8× 2.5" SAS/NVMe | 8× NVIDIA A100/H100 or AMD MI300X |
| PowerEdge XE9640 | 2U | 32×128 GB DDR5 (4 TB) | 4× 2.5" NVMe (61.44 TB) | 4× NVIDIA H100 |
| PowerEdge XE8640 | 4U | 32×128 GB DDR5 (4 TB) | 8× E3.S or 8× 2.5" SAS/NVMe | 4× NVIDIA H100 |
| PowerEdge R760xa | 2U | 32×256 GB DDR5 (8 TB) | 8× 2.5" SAS/NVMe | 4× 400W or 12× 75W |

**Definition**

**Rack Unit (U):** A standard unit of measurement for rack-mounted equipment height. 1U = 44.45 mm. A standard rack is 42U tall. Determines how many devices fit in one rack — critical for data center and edge deployment density.

- These are **server-class machines** — powerful enough for AI inference, HPC, deep learning
- Heavy GPU presence — edge computing increasingly means running **deep learning models at the edge**
- All use 5th Generation Intel Xeon Scalable processors

**Advantages of Edge/Fog Computing:**

- High computational capacity (server-class hardware)
- Distributed computing (processing spread across multiple edge nodes)
- Privacy and security (sensitive data does not leave the premises)
- Reduced latency (decisions made locally, not waiting for cloud round-trip)

**Disadvantages:**

- Require a power connection (not battery-powered like IoT)
- Require connection with the Cloud (for long-term storage and model updates)

**★ Exam Focus**

- **Know the three-layer pyramid**: IoT → Fog/Edge → Cloud and timescales at each level
- **Key motivation for Edge Computing**: latency — be able to explain with a concrete example
- **Design pattern**: Learning in Cloud, Deploying in Fog
- **Know the wind turbine example**: what each stage does and why the split between edge and cloud was made
- **Rack Unit (U)**: know the definition and why it matters for deployment density
- **Edge advantages**: low latency, high compute, privacy, distributed; **disadvantages**: needs power connection and cloud connectivity

# 9 Embedded PCs

## 9.1 What Are Embedded PCs?

- Full computers shrunk onto a single board
- Designed for **pervasive, embedded deployment** — inside machines, robots, vehicles
- Sit between IoT devices and edge servers on the spectrum
- Run full operating systems (typically Linux)
- Programmable like standard PCs

## 9.2 Key Examples from the Slides

- **Raspberry Pi 5** (8 GB, Quad-Core ARM A76, 64-bit, 2.4 GHz)
  - Most famous embedded PC
  - Runs full Linux
  - Cost: ∼$80
  - Complete computer the size of a credit card
  - Used everywhere from hobby projects to industrial control
- **Nvidia Jetson Orin Nano** (8 GB, 6×1.5 GHz)
  - Designed specifically for **AI at the edge**
  - Has dedicated neural network acceleration hardware
  - Used in robotics, autonomous systems, smart cameras

  - More expensive than Raspberry Pi but far more powerful for AI inference
- **BeagleBone Black** (ARM Cortex A8, 1 GHz, 512 MB RAM)
  - More hardware-oriented — many GPIO (General Purpose Input/Output) pins
  - Used in industrial control and hardware prototyping
  - Interface with sensors, motors, actuators directly
- **Google Coral Dev Board** (1 GB RAM)
  - Contains a **TPU (Tensor Processing Unit)** — Google's custom AI chip
  - Specifically for running TensorFlow Lite models at the edge with very low power

## 9.3 Advantages and Disadvantages

**Advantages:**

- **Pervasive computing** — can be embedded anywhere
- **High performance** unit for their size
- **Availability of development boards** — easy to prototype with
- **Programmed like PCs** — standard Linux, familiar tools, standard compilers
- **Large community** — vast online resources, libraries, support

**Disadvantages:**

- **Pretty high power consumption** for battery-powered scenarios (Raspberry Pi draws 5–8W, drains batteries quickly)
- **Some hardware design required** — often need custom PCBs, power regulators, enclosures for deployment

## 9.4 When to Use Which Infrastructure Type?

| maingreen!20 **Scenario** | **Infrastructure Choice** |
|---|---|
| Sense and transmit only | IoT device |
| Process locally with modest compute, run Linux | Embedded PC |
| Heavy AI inference or HPC locally | Edge Server |
| Massive scale compute or model training | Data Center / Cloud |

# 10 Internet of Things (IoT)

## 10.1 Definition

**Definition**

An **IoT (Internet of Things) device** is any everyday object embedded with **sensors, software, and internet connectivity** that allows it to **collect and exchange data** with other devices and systems, typically over the internet, with **limited need to process and store data**.

- IoT devices are primarily **data collectors and transmitters**, not processors
- They push raw or lightly-processed data upward to fog/cloud layers

## 10.2 Key Examples from the Slides

- **Arduino** (ATmega328, 16 MHz, 2 KB SRAM, 32 KB Flash)
  - Classic beginner microcontroller
  - No OS, no networking built-in
  - Bare-metal C programming
  - 2 KB of RAM — must count every byte
- **STM32** (ARM Cortex-M series)
  - Industrial-grade microcontroller used in real products
  - More powerful than Arduino, still resource-constrained
  - Used in appliances, automotive, industrial sensors
- **ESP32** (up to 240 MHz, 512 KB RAM)
  - Very popular — has **built-in WiFi and Bluetooth**
  - Most common IoT development platform today
  - Cheap (∼$5), wireless, reasonable compute for simple ML tasks
- **Particle Argon**
  - Cloud-connected development board with built-in WiFi and BLE
  - Designed for IoT product development with cloud integration built-in

## 10.3 Arduino Nano Family Specification Comparison

| maingreen!20 **Device** | **Microcontroller** | **Clock** | **SRAM** | **Flash** | |
|---|---|---|---|---|---|
| Arduino Nano | ATmega328 | 16 MHz | 2 kB | 32 kB | |
| Nano ESP32-S3 | ESP32-S3 | up to 240 MHz | 512 kB | 16 MB | |
| Nano 33 IoT | SAMD21 + u-blox | 48 MHz | 256 kB | 1 MB | |
| Nano 33 BLE | nRF52840 | 64 MHz | 256 kB | 1 MB | |
| Nano RP2040 Connect | RP2040 | 133 MHz | 264 kB | 16 MB | |
| Nano Every | ATMega4809 | 16 MHz | 6 kB | 48 kB | |
| Nano Matter | Silicon Labs MGM24 | 78 MHz | 256 kB | 1536 kB | |

- Even the most powerful Arduino Nano has **512 kB of RAM** at most
- Compare: Raspberry Pi has **8 GB RAM** = **16,000 times more**
- This RAM difference explains why IoT devices sit far left on the spectrum diagram
- I/O voltage: typically 3.3V (some 5V) — critical for interfacing with sensors

**Definition**

- **SRAM (Static RAM):** Working memory (like RAM in a PC) — very scarce in IoT; **lost on power off**
- **Flash:** Program storage (like an SSD) — where your code lives; **non-volatile**, survives power off

## 10.4 Advantages and Disadvantages of IoT

**Advantages:**

- **Highly pervasive** — can be embedded in any object
- **Wireless connection** — WiFi, BLE, Zigbee, LoRa, etc.
- **Battery powered** — can operate untethered for months or years
- **Low cost** — Arduino costs a few euros; enables mass deployment
- **Sensing and actuating** — interface directly with the physical world

**Disadvantages:**

- **Low computing ability** — cannot run complex algorithms; even simple neural networks are challenging
- **Constraints on energy** — battery life is a primary design constraint; every CPU cycle costs energy
- **Constraints on memory** — 2 KB RAM means you cannot load even a simple webpage; no complex data structures
- **Difficulties in programming:**

  - No OS abstractions, no virtual memory, no memory protection
  - A bug can crash the entire device with no recovery mechanism
  - Real-time constraints must be manually managed
  - No standard debugging tools; limited observability

**★ Exam Focus**

- **Know the IoT definition** and emphasize "limited need to process and store data"
- **SRAM vs. Flash**: SRAM is volatile working memory; Flash is non-volatile program storage
- **RAM comparison**: Arduino = 2 kB; Raspberry Pi = 8 GB = 16,000x difference — know this scale
- **IoT characteristics**: battery-powered, wireless, cheap, pervasive, severely constrained
- **IoT limitations**: low compute, memory constraints, energy constraints, programming difficulties (no OS, no memory protection)
- **Typical IoT chips**: Arduino (ATmega), ESP32, STM32, ARM Cortex-M series

# 11 The IT Perspective for Computing Infrastructures

## 11.1 The Three-Layer IT Model

**Definition**

Every computing infrastructure, regardless of type, can be described with three layers:

- **Hardware:** Physical resources of the system — computation, storage, input/output, networking
- **Software (Environment):** Programs and libraries to control the physical resources and provide tools to build applications — OS, device drivers, hypervisors, container engines, middleware
- **Software (Application):** The application, i.e., the reason this system exists — a web service, database, ML model, business application

**This model applies at every scale of infrastructure:**

- **On an Arduino:** HW = microcontroller, Environment = bare-metal runtime, Application = sensor reading loop
- **On a PC:** HW = Intel CPU, Environment = Windows/Linux, Application = browser

- **On a Data Center node:** HW = server rack, Environment = hypervisor + VMs, Application = cloud service

## 11.2 Mapping to Course Topics

| maingreen!20 **IT Layer** | **In the Cloud Context** | **Course Section** |
|---|---|---|
| Software (Application) | Cloud-Based Applications, X-Learning-as-a-Service | SW Infrastructures (Applications) |
| Software (Environment) | Cloud (Virtualization, IaaS/PaaS/SaaS) | SW Infrastructures (Virtualization & Cloud) |
| Hardware | Datacenter (Physical servers, storage, network) | HW Infrastructures |

**Definition**

**X-Learning-as-a-Service** (X-Learning-aaS): Using cloud APIs to access machine learning capabilities without managing any infrastructure. Example: calling OpenAI API, Google Vision API, AWS Rekognition. You consume AI as an application-level service.

- **This three-layer view is the mental model to carry throughout the entire course**
- When you study cooling systems — that is the hardware layer
- When you study Docker containers — that is the environment layer
- When you study cloud deployment patterns — that is the application layer being served by the layers below

# 12 Complete Spectrum Summary — The Big Picture

| maingreen!20 **Type** | **Compute** | **RAM** | **Latency** | **Location** | **Best For** |
|---|---|---|---|---|---|
| IoT | 0.0001–0.0005x | 0.01–0.05x | $\mu$secs | Everywhere | Sensing, actuating |
| Embedded PC | 0.05–0.1x | 0.1–0.5x | msecs | Embedded in machines | Local intelligence |
| PC | 1x | 1x | — | Desk/laptop | General purpose |
| Edge Server | 20–50x | 50–100x | 100ms–mins | On-premises/factory | Local compute, AI inference |
| Data Center | 100–1000x | Very high | mins–hours | Remote/cloud | Training, large-scale, storage |

**✓ Main Takeaway**

These infrastructure types are **not competing technologies** — they work **together in a hierarchy**:

- IoT collects data →
- Edge processes it locally →
- Cloud stores and learns from it →
- Models deployed back to Edge

The key engineering challenge is deciding **where to split the processing** across this hierarchy.

## 13 Master Exam Focus Summary — Lecture 1

**★ Exam Focus**

**Definitions to Know:**

- Computing infrastructure — technological infrastructure providing HW and SW to other systems/services
- IoT device — everyday object with sensors, software, internet connectivity

for data collection/exchange with limited processing

- Container — application packaged with dependencies sharing host OS kernel (no Guest OS)
- VMM/Hypervisor — software managing multiple isolated Guest OSes on one physical host
- UPS — Uninterruptible Power Supply; battery backup for servers during power failure
- Rack Unit (U) — 1U = 44.45 mm; standard for rack-mounted equipment height
- X-Learning-as-a-Service — accessing ML capabilities via cloud APIs without managing infrastructure

**Numbers to Memorize:**

- Data Center cost split: **45% servers, 25% infrastructure, 15% power, 15% network**
- 15 MW data center water use: **130 million gallons/year**
- Global installed power: **30 GW (2010) → 60 GW (2020)**
- US electricity: **1.9% (2014) → up to 12% (future projection)**
- 99.995% uptime = **Five Nines** = **≈26 minutes downtime/year**
- Arduino RAM: **2 kB**; Raspberry Pi RAM: **8 GB** = **16,000× difference**

**Comparisons to Master:**

- **VM vs. Container**: stack diagrams, Guest OS vs. shared host OS, isolation, overhead, startup time
- **Enterprise DC vs. Colocation vs. Managed Hosting vs. Cloud**: pros/cons of each
- **IaaS vs. PaaS vs. SaaS**: what you manage vs. what provider manages
- **Edge vs. Cloud**: latency, data volume, processing split decisions
- **IoT vs. Embedded PC vs. Edge Server**: compute power, RAM, use cases, programming model

**Concepts to Explain:**

- Why does Edge Computing exist? (latency — centralized DC cannot respond fast enough for real-time systems)
- The three-layer pyramid model (IoT → Fog/Edge → Cloud) with timescales
- The three-layer IT model (HW → SW Environment → SW Application) and how it applies to DC/Cloud
- The "build or not build" data center dilemma from a business perspective
- Why the PC is used as reference point (1x) in the spectrum diagram

- Design pattern: “Learning in Cloud, Deploying in Fog”
- Lock-in problem in managed hosting
- SRAM vs. Flash in IoT devices